



Universidad Nacional de Córdoba

Facultad de Ciencias Exactas, Físicas y Naturales

Compiladores y ejecutables

Práctica Supervisada

Informe de Trabajo

Supervisor:

Prof. Maximiliano Eschoyez

Tutor:

Nicolás Papp

Autores:

José Cancinos y ***Julián González***

2021

Resumen

El presente informe tiene como objetivo presentar el trabajo realizado para el cumplimiento de la asignatura Práctica Supervisada. En primer lugar, se presentan los aspectos generales sobre compiladores y la producción de código intermedio. Luego, se pone el foco en el análisis estructural de las **suites** de compilación: GNU GCC y CLANG/LLVM. Posteriormente, es detallado el proceso de generación y código resultante de cada etapa de los compiladores. Luego, se procede a una breve explicación sobre los archivos ejecutables, resultantes del proceso de compilación. Más tarde, se hace foco en los archivos de tipo `ELF`: propiedades y estructura. Finalmente, se exponen los distintos métodos de inserción y posterior lectura de datos dentro de un archivo ejecutable de formato `ELF`.
Los programas de inserción y lectura se emplearán como base para el trabajo de Proyecto Integrador, siendo ambos un inicio para un mejor desarrollo en cuanto a profundidad, abstracción y uso en espacio de sistema (*kernel*).

Índice general

1. Compiladores **1**
- I. Conceptos básicos sobre compiladores . . . 1
  - A. Concepto de Compilación . . . 1
  - B. Tipos de Compiladores . . . 1
  - C. Estructura de un Compilador . . . 1

2. Códigos intermedios **5**
- I. Conceptos sobre código intermedio . . . 5
  - A. Código de tres direcciones . . . 6

3. Compilación en GCC y CLANG/LLVM **10**
- I. Conceptos básicos sobre GCC . . . 10
  - A. Etapas del proceso de compilación en GCC . . . 10
- II. Conceptos básicos sobre CLANG/LLVM . . . 10
- III. Comparación entre las diferentes partes de los compiladores entre GCC y CLANG/LLVM . . . 10
  - A. Preprocesador . . . 10
  - B. Analizador léxico o *scanner* o *lexer* . . . 10
  - C. Analizador sintáctico o *parser* . . . 10
  - D. Analizador semántico . . . 11
  - E. Generador de código intermedio . . . 11
  - F. Optimizador . . . 13
  - G. Generador de código ensamblador . . . 13

4. Comparación entre GCC y CLANG/LLVM: Códigos **17**
- I. Código fuente . . . 17
- II. Etapas . . . 18
  - A. Preprocesamiento . . . 18
  - B. Analizador léxico y sintáctico . . . 18
  - C. Analizador semántico - fin del *frontend* . . . 19
  - D. Generador de código intermedio . . . 19
  - E. Optimizador . . . 20
- III. Código máquina . . . 20

5. Código ejecutable **21**
- I. Formatos de ejecutables en Unix . . . 21
- II. Formato ELF . . . 21
  - A. Información básica de un archivo ELF . . . 21
  - B. Estructura de un archivo ELF . . . 22

6. Inserción de información en archivo ELF **25**
I. Secciones ELF en donde colocar payload 25
A. Sección `.comment` 25
B. Sección *Read only data* 27
C. Nueva sección creada 28

7. Lectura de metadata insertada en archivo ELF **29**
I. Lectura ELF 29

8. Conclusión **31**

Índice de figuras

1.1. Fases de compilación. 3
1.2. Ejemplo completo de fases de compilación. [1] 4

2.1. Localización del código intermedio dentro de las etapas de un compilador. 6
2.2. Ejemplo de código de tres direcciones, cuadrúplas y triples. [2] 8
2.3. Ejemplo de código de tres direcciones tipo triples indirectos, SSA y *Minimal* SSA. [2] 8

3.1. Proceso de compilación en GCC. [3] 11
3.2. Estructura del compilador GNU GCC. [4] 11
3.3. Fases del compilador CLANG/LLVM. 13

5.1. Estructura de un archivo ELF 22

Índice de tablas

1.1. Clasificación de compiladores. 2

2.1. Clasificación de códigos intermedios 6
2.2. Distintos formatos de códigos intermedios dado diferentes compiladores. 7
2.3. Tipo de operaciones en código de tres direcciones y su formato. 7
2.4. Derivaciones de códigos de tres direcciones. 9

3.1. Etapas de compilación en GCC. 12
3.2. Módulos y etapas de compilación en CLANG/LLVM. 13
3.3. Comparación con respecto a preprocesador. 14
3.4. Comparación con respecto a analizador léxico. 14
3.5. Comparación con respecto a analizador sintáctico. 14
3.6. Comparación con respecto a analizador semántico. 15
3.7. Comparación del generador de código intermedio - *Middle-end*. 15
3.8. Comparación con respecto a optimizador. 16
3.9. Comparación con respecto a generador de código ensamblador. 16

5.1. Principales formatos de códigos ejecutables en Unix. 22

Listings

4.1. `ejemplo.c`. [5] . . . 17
4.2. Comando de compilación del archivo `ejemplo.c` [5] para GCC. . . . 17
4.3. Comando de compilación del archivo `ejemplo.c` [5] para CLANG/LLVM. . . . 17
4.4. Salida del preprocesador en GCC - `ejemplo.i`. [6] . . . 18
4.5. Salida del preprocesador en CLANG - `ejemplo.i`. [7] . . . 18
4.6. CLANG/LLVM `tokens.c`. [8] . . . 18
4.7. Código intermedio obtenido en GCC - `ejemplo.c.005t.gimple`. [9] . . . 19
4.8. Asignación de variables en LLVM IR de CLANG/LLVM - `ejemplo.ll`. [10] . . . 19
4.9. Pseudocódigo de la asignación de variables en LLVM IR - `ejemplo.ll`. [10] . . . 19
4.10. Código ensamblador generado por GCC - `ejemplo.s`. [11] . . . 20
5.1. Salida del comando *file* al ejecutable ELF `simpleEjGCC`. . . . 21
5.2. Comando `dump` sobre el archivo ELF `simpleEjGCC`. . . . 21
5.3. Lectura de la cabecera ELF usando el comando `readelf` sobre el archivo `simpleEjGCC`. . . . 22
5.4. Algunas líneas del mapeo entre secciones y segmentos del archivo ELF `simpleEjGCC`. . . . 23
5.5. Algunos *headers* de segmentos del archivo ELF `simpleEjGCC`. . . . 24
5.6. Algunos *headers* de sección del archivo ELF `simpleEjGCC`. . . . 24
6.1. Sección de comentario dentro del ELF `simpleEjGCC`. . . . 25
6.2. Lenguaje ensamblador para insertar datos en sección `.comment` en `simpleEjInline`. . . . 25
6.3. Lectura de la sección `.comment` del archivo `simpleEjInlineGCC`. . . . 26
6.4. Lectura de la sección `.comment` del archivo `simpleEjInlineGCC` después de realizar copia de la información. . . . 26
6.5. Inserción de payload en sección `.comment` usando `atributte`. . . . 26
6.6. Sección `.comment` usando `attribute` para insertar payload. . . . 27
6.7. Inserción de payload en sección `.rodata` - `cod_rodata.c`. . . . 27
6.8. Sección `.rodata` con payload agregado. . . . 27
6.9. Inserción de payload en sección `.metadata` en ejecutable `macrosCommentGCC`. . . . 28
6.10. Comando de compilación para pasar el *string* (*payload*) como variable de entorno al compilador. . . . 28
6.11. Contenido de la sección `.metadata`. . . . 28
7.1. Carga del archivo ELF en código `lecturaELF.c`. . . . 29
7.2. Lectura y almacenamiento de la tabla de símbolos en `lecturaELF.c`. . . . 29
7.3. Lectura y almacenamiento de la tabla de *strings* en código `lecturaELF.c`. . . . 29
7.4. Almacenamiento e impresión de los datos que contiene la sección buscada. Código del programa en `lecturaELF.c`. . . . 30
7.5. Ejecución del programa `lecturaELF`. . . . 30

Capítulo 1

Compiladores

I Conceptos básicos sobre compiladores

A *Concepto de Compilación*

Compilación: Conjunto de procesos efectuados para obtener un archivo ejecutable a partir de código fuente. [12]

B *Tipos de Compiladores*

Los siguientes conceptos son fundamentales para introducir la clasificación.

- ***Host***: arquitectura de la máquina en la que va a correr el compilador.
- ***Build***: arquitectura que se usa para generar el compilador.
- ***Target***: arquitectura en la que deberá correr el ejecutable generado por el compilador.

Así, se presenta la clasificación de los distintos tipos de compiladores (Von Hagen, [12]) en la tabla 1.1.

C *Estructura de un Compilador*

Las fases que componen un compilador [1] pueden verse en la Fig. 1.1.

1) Analizador léxico

El analizador léxico, también llamado *scanner* o *lexer*, es aquel que lee la secuencia de caracteres del código fuente y los agrupa en otras secuencias llamadas **lexemas**. Por cada lexema, el *scanner* genera un *token* de la forma: La Fig. 1.2 muestra las entradas y salidas de la fase en la parte superior.

$$< \textit{nombre-token}, \textit{valor-atributo} > \tag{1.1}$$

Siendo,

- **nombre-token**: símbolos abstractos usados durante el análisis sintáctico.
- **valor-atributo**: puntero a una entrada en la tablas de símbolos.

Tabla 1.1: Clasificación de compiladores.

Tipo de compilador	Descripción
Compilador nativo	Compilador que genera ejecutables para el mismo tipo de sistema en el cual está operando. (Ídem *build*, *host* y *target*)
Compilador "cruzado"	Llamado *cross-compiler* en inglés, compilador que corre en una arquitectura específica y genera código para otra distinta. (idem *build* y *host*, distinto *target*)
Crossback compiler	El sistema en donde corre y el de ejecución de binarios son iguales pero el sistema *host* es distinto. (idem *build* y *target*, distinto *host*). Se usan para construir un *cross-compiler* que corra en el sistema en el que el compilador corre)
Crossed native compiler	El sistema *target* y el *host* son los mismos pero el sistema en donde se construye el compilador es distinto. Usa un *cross-compiler* para construir un compilador nativo en un tercer sistema. (idem *target* y *host*, distinto *build*)
Compilador canadiense	El sistema en donde se construye, el *host* y el de destino son todos distintos. Un compilador que construye sobre una arquitectura, corre en otra arquitectura y crea código para una tercera arquitectura. (*build*, *host* y *target* son distintos)

Tabla de símbolos El compilador guarda los nombres de variables o nombres de funciones (que aparecen en el código fuente) en la **tabla de símbolos**. También, almacena atributos varios de dicha variable, por ejemplo: tipo, *scope*, argumentos, tipo de pasaje de argumentos, tipo de retorno, etc. Luego, se realiza un mapeo de *tokens* con sus respectivos nombres de la tabla.

2) Analizador sintáctico

El analizador sintáctico, también llamado *parser*, utiliza el primer componente de cada *token* (*nombre-token* en ecuación 1.1) para crear una representación en forma de árbol que muestre la estructura de los *tokens*. Se genera un **árbol sintáctico**. La Fig. 1.2 refleja un esquema de esta fase en la parte superior.

3) Analizador semántico

El analizador semántico emplea el árbol sintáctico y la tabla de símbolos (creada por el *scanner*) para revisar la consistencia semántica del código fuente con respecto a la definición del lenguaje. También se realizan conversiones de un tipo de dato a otro (llamadas coerciones). Ver Fig. 1.2 en la fase correspondiente.

4) Generador de código intermedio

El generador intermedio tiene como meta producir código de tipo intermedio para obtener un lenguaje flexible y optimizable para la parte *backend* del compilador. Observar dentro de la Fig. 1.2 para una ilustración de la fase.

El código intermedio puede tener muchas formas distintas (dependiendo de cada compilador).

Propiedades de todo código intermedio Todo código intermedio debe cumplir las propiedades de:
a) ser fácil de generar; *b*) ser fácil de traducir a lenguaje máquina

Código de tres direcciones Es una forma de código intermedio que consiste en una secuencia de instrucciones cuasi-*assembler* con tres operandos por cada instrucción (cómo máximo). Cada operando puede actuar como un registro.

Algunas características de este tipo de código son:

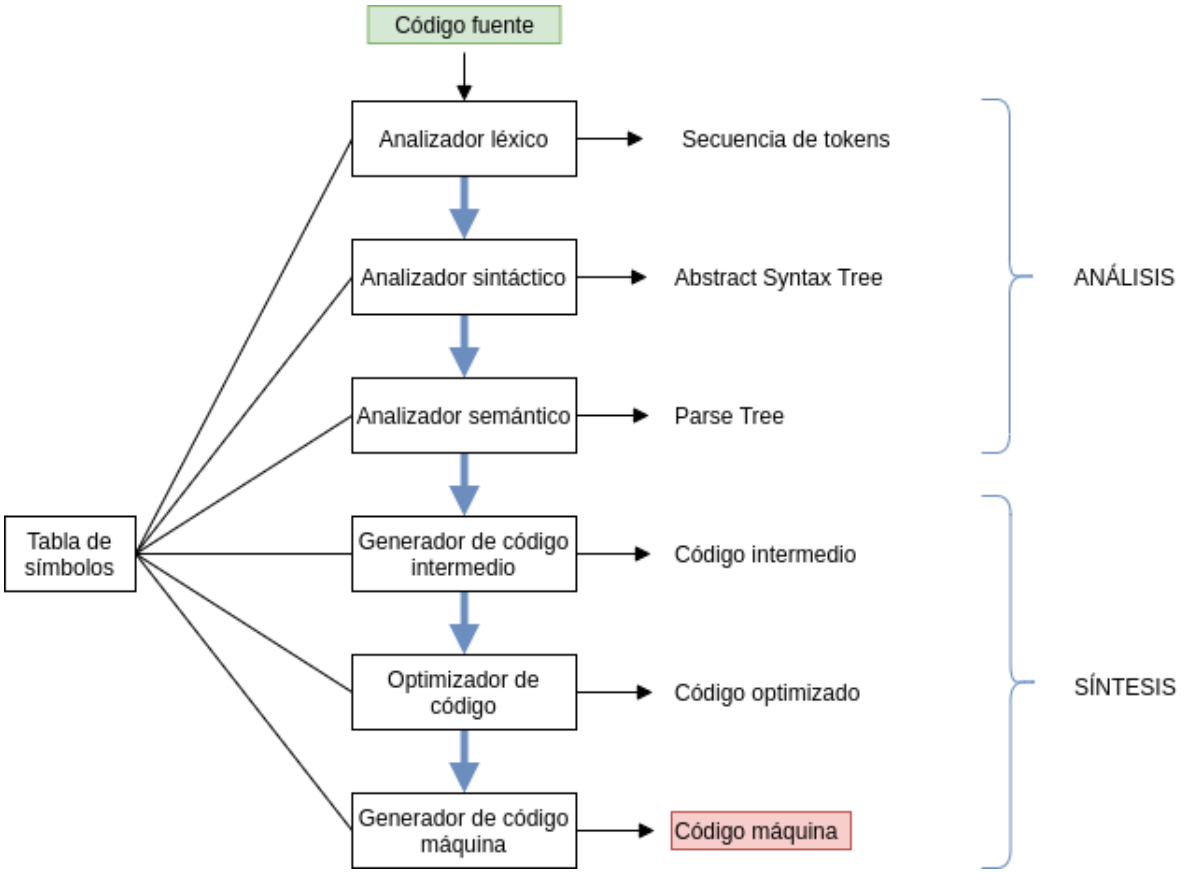


Figura 1.1: Fases de compilación.

- Las instrucciones deben tener como máximo un **único operador** (órden de operaciones)
- El compilador debe generar un **nombre temporal** para guardar el valor retornado por la instrucción de tres direcciones (variables temporales tienen nombre).

Mayor detalle sobre código intermedio es presentado en el Cap. 2.

5) Optimizador de código intermedio

El optimizador de código intermedio realiza mejoras sobre dicho código para obtener un “mejor código” como salida de cada sucesiva optimización. Se presenta también una ilustración en la parte inferior de la Fig. 1.2.

Mejor código = más rápido, más corto o de menor consumo de potencia.

6) Generador de código final

El generador de código final utiliza el código optimizado para generar el código final deseado. Si el lenguaje final es código máquina, se eligen los registros o lugares de memoria para cada variable usada en el programa. En este trabajo, el código final buscado es código máquina. Por ello, se empleará el término "generador de código máquina". En este caso, se traducen instrucciones inmediatas a secuencias de instrucciones máquina. La parte inferior de la Fig. 1.2 muestra una ilustración sobre esta fase.

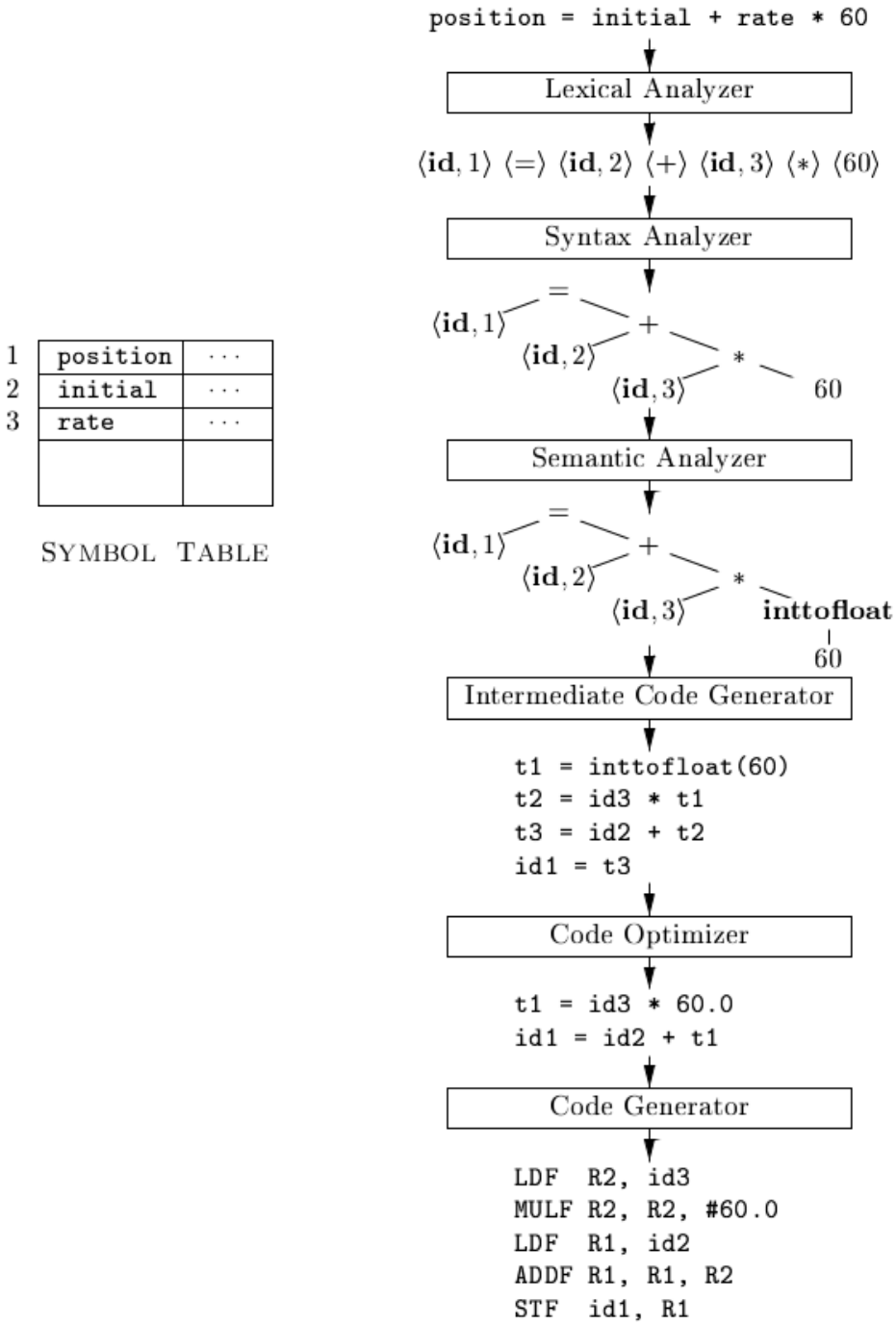


Figura 1.2: Ejemplo completo de fases de compilación. [1]

Capítulo 2

Códigos intermedios

I Conceptos sobre código intermedio

1) Definición

El código intermedio, también llamado **representación intermedia** (IR - *Intermediate Representation*), es el código generado entre las etapas de análisis (*frontend*) y síntesis (*backend*) de la compilación.

Es generado por la salida del *frontend*. El optimizador realiza diversas optimizaciones sobre el código intermedio obtenido. El *backend* utiliza el código intermedio final (ya optimizado) para obtener el código máquina.

2) Características de todo código intermedio

Todo código intermedio cuenta con las siguientes características principales:

- No hay un único código intermedio. Éste varía de compilador a compilador.
- Debe ser un lenguaje definido o un conjunto de estructuras de datos compartidas entre las fases de compilación.
- Debe ser portable. Debe soportar *frontends* de distintos lenguajes fuente así como abarcar distintos *backends* para código máquina diversos.
- Debe ser fácilmente optimizable, a través de diversas iteraciones sin demasiada complejidad.
- Puede ser capaz de ser interpretado directamente.

3) Clasificación de códigos intermedios

Todo código intermedio pertenece a alguna de las categorías expuestas en la tabla 2.1.

4) Ejemplos de códigos intermedios

La tabla 2.2 presenta ejemplos de los distintos tipos de códigos intermedios que emplean diferentes compiladores.
La tabla 2.2 presenta ejemplos de los distintos tipos de códigos intermedios que emplean diferentes compiladores.

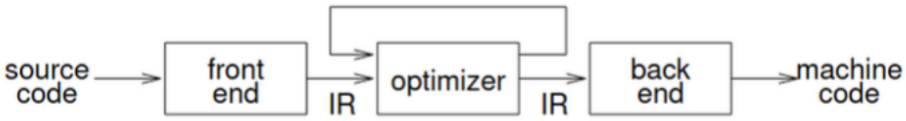


Figura 2.1: Localización del código intermedio dentro de las etapas de un compilador.

Tabla 2.1: Clasificación de códigos intermedios

Estructurales	**Lineales**	**Híbridos**
▪ Son orientados a la forma gráfica. ▪ Son usados principalmente para traducir código fuente en las etapas iniciales de la compilación. ▪ Sus estructuras (nodos y vértices) son muy grandes. ▪ Ejemplos: árboles, DAG.	▪ Son parecidos a un pseudocódigo de código máquina. ▪ Son simples y compactos. ▪ Son fáciles para reorganizar el código (más simple optimización) ▪ Ejemplos: RTL, GIMPLE, LLVM IR, etc.	▪ Son una combinación de grafos y código lineal. ▪ Ejemplos: CFG (*control flow graphs*) en GCC.

A *Código de tres direcciones*

Código donde cada instrucción tiene como máximo un operador.

Estructura

$$< x = y \; op \; z > \tag{2.1}$$

Siendo,

- x : resultado.
- y : operando 1.
- z : operando 2.
- op : operación.

Instrucciones y direcciones

El código de tres direcciones se basa en dos conceptos: direcciones e instrucciones.

Direcciones: Las direcciones pueden ser:

- Nombres. En implementación, son reemplazados por un puntero a una entrada de la tabla de símbolos.
- Constantes.
- Variables temporales generados por el compilador. Siempre se usan variables temporales. No puede asignarse una variable temporal más de una vez.

Tabla 2.2: Distintos formatos de códigos intermedios dado diferentes compiladores.

Compiladores	**Formato de código intermedio adoptado**
JVM	Java bytecode
LLVM/CLANG	SSA y TAC
GCC	Distintos formatos de IR: AST (*Abstract syntax trees*), GENERIC (basado en árboles), GIMPLE (tipo SSA) y RTL (basado en LISP)
.Net Compiler	Microsoft CIL

Tabla 2.3: Tipo de operaciones en código de tres direcciones y su formato.

Tipo de instrucciones	**Formato**
De asignación	$x = y\ op\ z$
Unarias: incremento, decremento, negación, casteo de tipo	$x = op\ y$
Copia	$x = y$
Saltos incondicionales	$goto\ L$
Saltos condicionales	$if\ x\ goto\ L$
Saltos condicionales con operadores condicionales	$if\ x\ relop\ y\ goto\ L$
Llamada o retorno de funciones	$return\ y \quad call\ p,\ n$
Copias indexadas	$x = y[i] \quad x[i] = y$
Asignaciones de direcciones y punteros	$x = \&y \quad x = *y \quad *x = y$

Instrucciones: La tabla 2.2 presenta los distintos tipos de instrucciones y formatos correspondientes para código intermedio.
Las etiquetas simbólicas son usadas por las instrucciones que alteran el flujo de control del código. Representan el índice de cada instrucción de tres direcciones en una secuencia de instrucciones.

Un ejemplo de este tipo de código intermedio se refleja en la Fig 2.2 a.

*1) Código de tres direcciones (*TAC - Three Address Code*)*

La tabla 2.4 presenta una comparativa entre las distintas representaciones que se han desarrollado a partir del código de tres direcciones estándar.

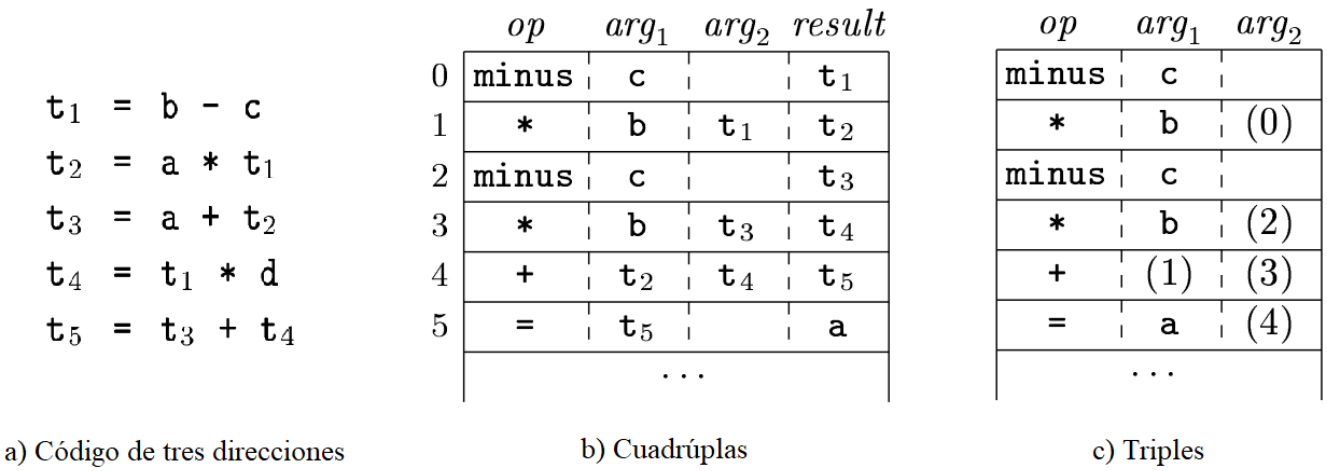


Figura 2.2: Ejemplo de código de tres direcciones, cuadrúplas y triples. [2]

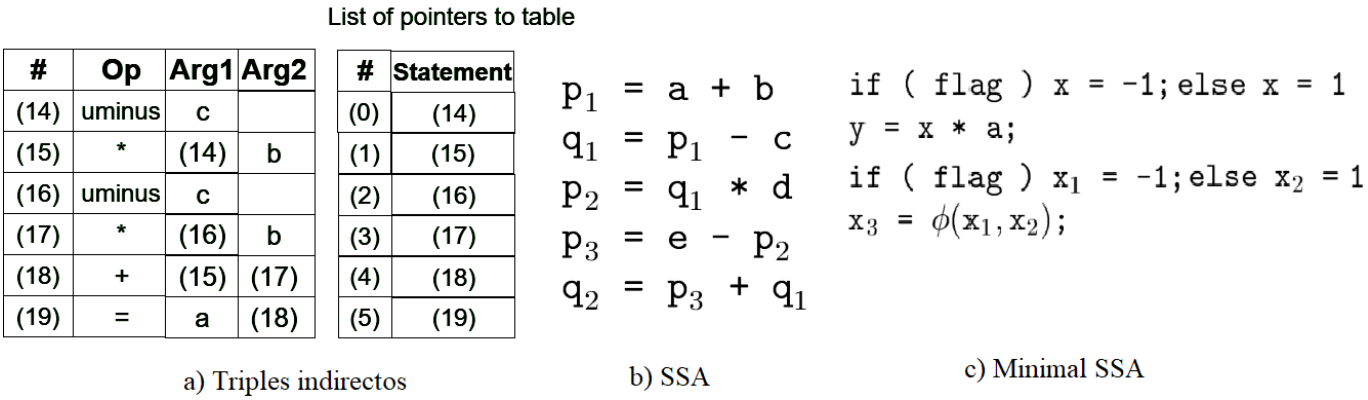


Figura 2.3: Ejemplo de código de tres direcciones tipo triples indirectos, SSA y *Minimal* SSA. [2]

Tabla 2.4: Derivaciones de códigos de tres direcciones.

Cuadrúplas	Triples	Triples indirectos	SSA - *Static single assignment*
Instrucciones con cuatro campos ▪ *op* (opcode); ▪ *arg* 1; ▪ *arg* 2; ▪ *resultado*. Contienen restricción de asignación única de variables.	Instrucciones que tienen tres campos. Ellos son: *op*; *arg* 1 y *arg* 2. Utilizan punteros para referenciar el resultado de una operación en lugar de guardar en variables temporales.	Consisten en una lista de punteros a triples, en lugar de una lista de triples en sí misma. Permite, entonces, que se pueda mover una instrucción reordenando la lista de instrucciones.	Las instrucciones contienen variables con nombres distintos. Toda variable solamente puede ser asignada por **única** vez. A su vez, éstas pueden ser leídas indefinidamente varias veces. Existe una versión llamada *Minimal* SSA, que utiliza una notación llamada ϕ-*function* para combinar dos definiciones de una misma variable en dos distintos flujos de código.
Ventajas: 1. Buena optimización al ser fácil el reordenamiento del código usando cuadrúplas. 2. Rápido acceso a variables temporales.	No hacen uso de variables para guardar resultados de operación.	Se puede mover una línea de instrucción cambiando el lugar de entrada en la tabla de punteros sin perder las referencias en la tabla de símbolo.	Mejor optimización en compilación, no se debe chequear que la asignación se ha realizado más de una vez.
Desventajas: 1. Generan una gran temporales, aumentando el tamaño de la tabla de símbolos. 2. La creación de variables temporales incrementa el tiempo de traducción.	Si se mueve una línea de instrucción que define una variable temporal, se cambian todas las referencias de índice. Genera complejidad alta. Por ello, generalmente no se usan triples en la optimización.	Uso de una nueva tabla (tabla de punteros).	Mayor cantidad de variables. Mayor tamaño de la tabla de símbolos.

Capítulo 3

Compilación en GCC y CLANG/LLVM

I Conceptos básicos sobre GCC

GCC (*GNU Compiler Collection*) es el grupo de compiladores usados por defecto *Linux* con `gcc` para C y `g++` para C++. GCC realiza los pasos de preprocesamiento, compilación, ensamble y enlace (*link*).

A Etapas del proceso de compilación en GCC

Las etapas que conforman a la *suite* de compilación GNU GCC es visualizado en la Fig 3.1.

II Conceptos básicos sobre CLANG/LLVM

LLVM (*Low Level Virtual Machine*) es una *suite* de compilación que admite múltiples lenguajes como C, C++, Objective-C, Fortran, Ada, Haskell, Java bytecode, Python, Ruby, ActionScript, GLSL, Clang, Rust, Gambas, etc.

El proyecto LLVM mantiene separadas las fases de *frontend* y *backend* de compilación. CLANG es el *frontend* mientras que LLVM es el *backend*.

III Comparación entre las diferentes partes de los compiladores entre GCC y CLANG/LLVM

Se describirán con detalle los elementos de entrada y salida de cada parte de compilación en GCC, tal y como se ha definido en INSERTAR REFERENCIAS. La Fig. 3.2 brinda una representación visual de las partes a explicar. Lo mismo sucederá con la suite de compilación CLANG/LLVM. La figura 3.3 muestra la estructura de la *suite* CLANG/LLVM.

A Preprocesador

Comparación en tabla 3.3.

B Analizador léxico o scanner *o* lexer

Diferencias se muestran en tabla 3.4.

C Analizador sintáctico o parser

La comparativa se presenta en tabla 3.5.

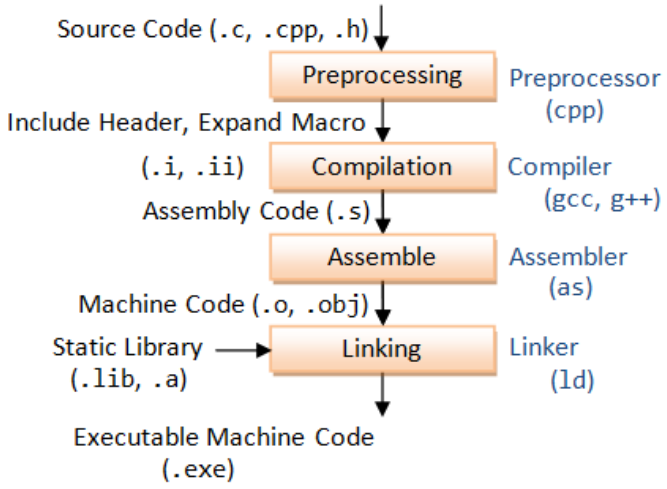


Figura 3.1: Proceso de compilación en GCC. [3]

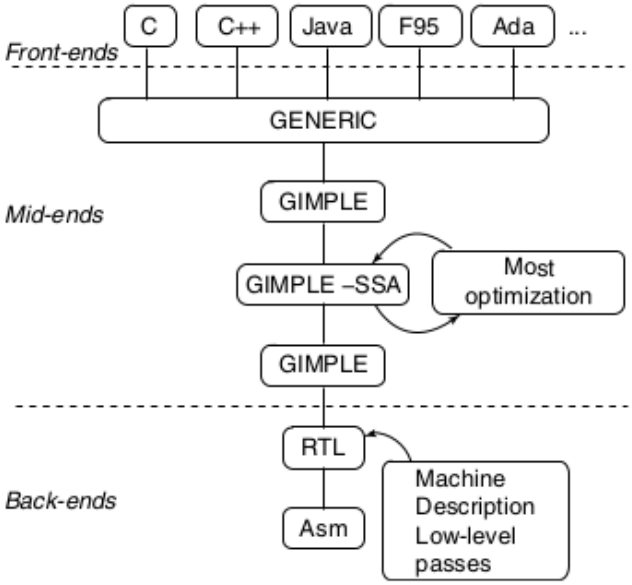


Figura 3.2: Estructura del compilador GNU GCC. [4]

D Analizador semántico

Las diferencias con respecto a la fase de análisis semántico se divisan en tabla 3.6.

E Generador de código intermedio

La comparación entre ambos generadores de código intermedio se presenta en la tabla 3.7. A su vez, a continuación se presenta mayor detalle de los códigos intermedios de ambas *suites* de compilación.

GCC GIMPLE

El formato GIMPLE es el elegido como código intermedio principal en GCC. Es posible obtener el código GIMPLE al realizar un `dump` en la compilación (mientras que GENERIC no permite esta opción).

Tiene las siguientes características:

- Es un código intermedio de tipo 3 direcciones.
- Es resultante de desglosar *GENERIC* en tuplas de no más de 3 operandos, a través de la herramienta interna *gimplifier* (excepciones: llamadas a funciones).
- Contiene variables temporales introducidas para poder computar expresiones complejas.
- Permite control de flujo de control a nivel inferior (el programa ahora se compone de sentencias secuenciales y de saltos incondicionales)
- El *scope* de cada función es local, simple.
- Es una representación conveniente para optimizar el código fuente.

Tabla 3.1: Etapas de compilación en GCC.

Etapa	Descripción
Preprocesamiento	Se toma el código fuente y se quitan todos los comentarios. Se incluye el código de los archivos de cabecera (*headers*) y se reemplazan todos los macros por sus valores. **Entradas**: Código en C (`.c` o `.h`). **Salidas**: Archivo de formato `.i`.
Compilación	Etapa explicada con mayor detalle en el título III. **Entradas**: Archivo `.i` preprocesado. **Salidas**: Código en lenguaje ensamblador `.s`.
Ensamblado	Este paso implica tomar la salida del *backend* del compilador y la transformación en código objeto, es decir, código en lenguaje máquina. **Entradas**: Código ensamblador `.s`. **Salidas**: Archivo objeto `.o`.
Enlace	En la etapa de enlace se crea el archivo final ejecutable. Esta fase tiene dos roles: 1. Generar el enlace de todos los archivos objetivos juntos, eso es, todos los otros códigos fuente en el proyecto. 2. Enlazar las llamadas a funciones con sus definiciones.. **Entradas**: Archivo objeto `.o` y librerías: estáticas (`.a`), compartidas (`.so`, en *Linux*) o utilizando un *flag* con el nombre de la librería (`-lNOMBRE`) **Salidas**: Archivo ejecutable (`.out` u otro formato ejecutable, generalmente ELF) o librerías compartidas (`.so`, empleando el *flag* `-shared`)

Las distintas partes del *frontend* de C y C++ actualmente convierten desde el árbol AST de *frontend* (*ORIGINAL*) a *GIMPLE*, sin pasar por *GENERIC*.

Existen 3 tipos de *GIMPLE*:

1. GIMPLE alto nivel: es el primer resultado de manipulación desde *GIMPLE*.
2. GIMPLE bajo nivel: se obtiene linealizando todas las estructuras de control de alto nivel del *GIMPLE* alto nivel, incluyendo funciones anidadas, manejo de excepciones y *loops*.
3. GIMPLE SSA: es *GIMPLE* bajo nivel en la forma *SSA* (*Static Single Assignment*)

CLANG/LLVM LLVM IR

El código intermedio usado en LLVM/CLANG es llamado *LLVM IR Assembly* o simplemente LLVM IR.
Éste contiene las siguientes características:

- Es de tipo tres direcciones SS, es decir, cada registro es asignado una única vez.
- Es independiente de la plataforma.
- No sólo es una representación en memoria (*in-memory*) sino que también se guarda en el disco (*dump*). Ésto es muy útil ya que GCC no hace ésto último.

Formas equivalentes de LLVM IR Existen tres tipos de presentar el LLVM IR. Todos son equivalentes y pueden pasar de uno a otro de forma simple.

1. Representación en memoria (clase *Instruction*)
2. Representación en disco codificada (*Bitcode* "`.bc`")
3. Representación en disco en forma de texto legible (*readable*) para humanos (*LLVM IR Assembly* "`.ll`")

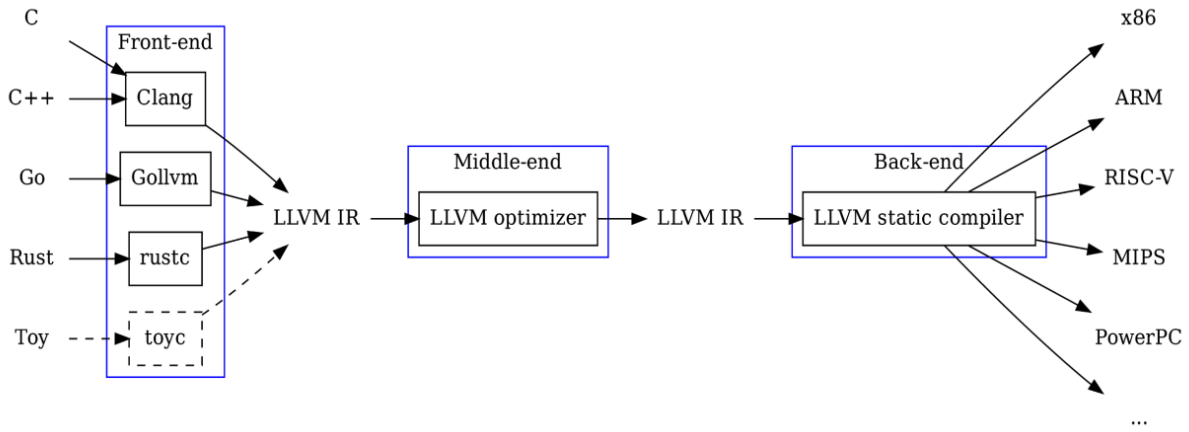


Figura 3.3: Fases del compilador CLANG/LLVM.

Tabla 3.2: Módulos y etapas de compilación en CLANG/LLVM.

Módulos	**Descripción**
Frontend	El módulo *frontend* del proyecto es **CLANG**. Es el paso del compilador que traduce el código fuente (C/C++/ObjectiveC) a código LLVM IR. Incluye las etapas de: 1. Preprocesador. 2. Analizador léxico. 3. *Parser* sintáctico. 4. Analizador semántico, y 5. Generador de código LLVM IR. **Entradas**: Código en C (`.c` o `.h`). **Salidas**: Código de formato código AST.
Middle-end	Este módulo tiene a cargo la generación, manipulación y optimización de código intermedio. Presenta las etapas de: 1. Generador de código intermedio y 2. Optimizador de código intermedio. **Entradas**: Código AST. **Salidas**: Código LLVM IR optimizado.
Backend	Es el paso responsable de la generación de código. Convierte LLVM IR a código *assembler* específico de arquitectura o binarios. Incluye: 1. Asignación de registros. 2. Transformaciones de *loops*. 3. Optimizadores de *peepholes* y 4. Optimizaciones específicas de arquitectura. **Entradas**: Código LLVM IR optimizado. **Salidas**: Código ensamblador `.s` o archivo objeto `.o`.

F Optimizador

Las diferencias con respecto a la fase de optimización se exponen en tabla 3.8.

G Generador de código ensamblador

Finalmente, la comparación de la última etapa: generación de código ensamblador, se presenta en la tabla 3.9.

Tabla 3.3: Comparación con respecto a preprocesador.

Compilador	GNU GCC	CLANG/LLVM
Descripción	Etapa a cargo del preprocesador de GNU GCC, **cpp**	Ejecutada por la clase Preprocessor, componente del *frontend* de CLANG
Funciones	Se toma el código fuente y se quitan los comentarios, se incluye el código de los archivos de cabecera (*headers*) y se reemplazan todos los macros por sus valores.	Ídem
Entradas	Código en lenguaje C, `.c` o `.h`.	Ídem
Salidas	Código extendido `.i`.	Ídem

Tabla 3.4: Comparación con respecto a analizador léxico.

Compilador	GNU GCC	CLANG/LLVM
Descripción	Etapa realizada por la herramienta **Flex**, *scanner*/*lexer* que no forma parte de GNU, pero es usado junto a GNU Bison (*parser*).	Realizado por la clase Lexer, parte del módulo *frontend* de CLANG.
Funciones	1. Leer el código extendido y realizar su *tokenización*. 2. Funcionar paralelamente con *parser*. El *scanner* es llamado cuando el *parser* necesita un nuevo *token*.	Ídem
Entradas	Código extendido `.i`. Archivo `lex.yy.c` por cada secuencia de *tokens*. NOTA: Es un archivo interno al proceso de compilación. GCC no lo almacena (***no dump***)	Código extendido `.i`, como punto de entrada si no se usa el preprocesador anteriormente. Tokens
Salidas	*Tokens* (***no dump***)	Llamados *Tokens* normales. (***no dump***)

Tabla 3.5: Comparación con respecto a analizador sintáctico.

Compilador	GNU GCC	CLANG/LLVM
Descripción	Etapa llevada a cabo por **Bison**, el generador de *parser* de GNU.	Ejecutada por la clase Parser, dentro del módulo *frontend* de CLANG.
Funciones	1. Chequear si la gramática del lenguaje acepta la secuencia de *tokens* generados por el *lexer*. 2. Reportar errores de sintáxis. 3. Producir el *error recovery*, si es posible. El funcionamiento de *lexer-parser* se realiza de forma jerárquica, con el siguiente orden: *1* Funciones; *2* Líneas internas de función; *3* Palabras.	Ídem
Entradas	Archivo `parse.y` (***no dump***) por cada secuencia de *tokens*.	*Tokens* normales
Salidas	*Tokens* de salida de parser (***no dump***)	*Tokens* con anotaciones (*Annotation tokens*, en inglés)

Tabla 3.6: Comparación con respecto a analizador semántico.

Compilador	GNU GCC	CLANG/LLVM
Descripción	Etapa realizada por **Bison**, el generador de *parser* de GNU.	Ejecutada por la clase Sema, presente en el módulo *frontend* de CLANG.
Funciones	Crear la representación en forma de árbol de la estructura sintáctica del código.	Ídem
Entradas	▪ Código fuente `.c`. ▪ Archivos *tokens* resultantes de *lexer* y *parser*.	Ídem.
Salidas	Archivo `.ORIGINAL`, representación del AST (*abstract syntax tree*), árbol sintáctico abstracto.	Archivo AST (árbol abstracto sintáctico). CLANG permite observar el AST de dos formas distintas: 1. Como código más cercano al modo en que es *parseado* el código fuente. (*Flag* `-ast-print`) 2. Como código en representación parecida a LISP. (*Flag* `-ast-dump`)

Tabla 3.7: Comparación del generador de código intermedio - *Middle-end*.

Compilador	GNU GCC	CLANG/LLVM
Descripción	Existen distintos pasajes del código intermedio en GCC. Por defecto, el *frontend* para compilación en C y C++ onvierte desde el árbol AST de *frontend* (*ORIGINAL*) a *GIMPLE*, sin pasar por *GENERIC*.	El código intermedio usado en LLVM/CLANG es llamado *LLVM IR Assembly* o simplemente LLVM IR.
Funciones	Tomar el código generado por el analizador semántico y transformarlo en código intermedio.	Ídem
Entradas	Código en formato *.ORIGINAL*	Código AST.
Salidas	Código intermedio de tipo GIMPLE. Más específicamente, *low level GIMPLE*.	Código intermedio de tipo LLVM IR, con formatos: ▪ Representación en memoria, interno al compilador. ▪ Bitcode `.bc`. ▪ Readable `.ll`.

Tabla 3.8: Comparación con respecto a optimizador.

Compilador	GNU GCC	CLANG/LLVM
Descripción	Forma parte del *backend* de GCC. El optimizador de GCC utiliza un código intermedio de muy bajo nivel (*RTL*). Las instrucciones son llevadas, una por una, a una forma algebráica que describe la acción. No es usado como código intermedio ya que no permite extraer información de alto nivel, por ejemplo: referencias a arreglos o tipo de datos.	Todavía forma parte de la etapa *middle-end* de la *suite* CLANG/LLVM. El optimizador realiza sucesivas iteraciones que generan un código totalmente optimizado a nivel de *frontend* y *middle-end*.
Funciones	▪ Generar *RTL* y, luego, optimizar el mismo para obtener código de un nivel más bajo que *GIMPLE*. ▪ Introducir características de código de bajo nivel tales como registros, modos de direccionamiento de memoria, tamaño de palabras y tipos de palabras, instrucciones de *branch* y de comparación, convenciones de llamadas, operaciones de bit, etc.	Llevar a cabo los distintos *passes*, operaciones realizadas LLVM IR que implican un cambio en el código o cambio sobre información acerca del código. Existen dos tipos de *passes*: ▪ **De análisis**: operaciones sobre la información relacionada a una unidad de código LLVM IR sin cambiar dicho código. Generalmente, implican grafos. ▪ **De transformación**: transforman una unidad de código IR. Pueden depender de resultados de análisis e invalidan resultados de análisis cuando se realiza la transformación.
Entradas	Código `.GIMPLE`.	Código LLVM IR.
Salidas	Archivo de tipo *RTL* con notación `.final`.	LLVM IR de un nivel más bajo (todavía válido como código intermedio) y con mayor optimización.

Tabla 3.9: Comparación con respecto a generador de código ensamblador.

Compilador	GNU GCC	CLANG/LLVM
Descripción	Etapa final del proceso de generación de código ensamblador.	Conforma el *backend* de CLANG/LLVM, es el paso responsable de la generación de código. Convierte LLVM IR a código *assembler* específico de arquitectura o binarios.
Funciones	Obtención del código máquina orientado a una determinada arquitectura.	Ídem. Implica el paso por etapas bien definidas dentro de LLVM, éstas son: 1. Asignación de registros. 2. Transformaciones de *loops*. 3. Optimizadores de *peepholes*. 4. Optimizaciones específicas de arquitectura. Con ello, el paso por disintos códigos internos, siendo instrucciones de tipo instancias *MCInst* anterior a obtención de código ensamblador o binario.
Entradas	Código de tipo RTL, en particular, `.final`.	LLVM IR optimizado
Salidas	Código en lenguaje ensamblador `.s`.	▪ Código ensamblador `.s`. ▪ Objeto binario.

Capítulo 4

Comparación entre GCC y CLANG/LLVM: Códigos

A partir de un mismo código fuente en lenguaje C, se procederá a la compilación y obtención de código *assembler* empleando tanto GCC como CLANG/LLVM. Luego, se llevará a cabo una comparación y breve descripción de lo observado.

Se presentan los códigos relevantes que coinciden con las etapas nombradas anteriormente. Para observar el resto de los archivos generados por los compiladores, dirigirse al repositorio de trabajo [13].

I Código fuente

```c
int main() {
  int num_a = 3;
  int num_b = 4;

  num_b = num_a;

  return 0;
}
```

Listing 4.1: `ejemplo.c`. [5]

La compilación y obtención de los distintos códigos, que se presentan a continuación, se logra empleado los siguientes comandos.

Para el caso de GCC,

```
    $ gcc -fdump-tree-original-raw -fdump-tree-all-graph -fdump-tree-gimple -da -S ejemplo.c
    $ gcc -o ejemplogcc ejemplo.c
```

Listing 4.2: Comando de compilación del archivo `ejemplo.c` [5] para GCC.

Mientras que, para el caso de CLANG/LLVM,

```
    $ clang ejemplo.c -o ejemploclang
    $ clang -fsyntax-only -Xclang -dump-tokens ejemplo.c  2>&1 | tee tokens.txt
    $ clang -fsyntax-only -Xclang -ast-dump ejemplo.c -fno-color-diagnostics > ejemplo.ast
    $ clang ejemplo.c -emit-llvm -c -o ejemplo.bc
    $ clang ejemplo.c -emit-llvm -S -c -o ejemplo.ll
    $ llc ejemplo.ll -print-machineinstrs 2>&1 | tee machineinstruct.txt
    $ llc ejemplo.bc -asm-show-inst -o ejemplo.s
```

Listing 4.3: Comando de compilación del archivo `ejemplo.c` [5] para CLANG/LLVM.

```
# 1 "ejemplo.c"
# 1 "<built-in>"
# 1 "<command-line>"
# 31 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 32 "<command-line>" 2
# 1 "ejemplo.c"
int main() {
 int num_a = 3;
 int num_b = 4;

 num_b = num_a;

 return 0;
}
```

Listing 4.4: Salida del preprocesador en GCC - `ejemplo.i`. [6]

```
# 1 "ejemplo.c"
# 1 "<built-in>" 1
# 1 "<built-in>" 3
# 316 "<built-in>" 3
# 1 "<command line>" 1
# 1 "<built-in>" 2
# 1 "ejemplo.c" 2
int main() {
 int num_a = 3;
 int num_b = 4;

 num_b = num_a;

 return 0;
}
```

Listing 4.5: Salida del preprocesador en CLANG - `ejemplo.i`. [7]

II Etapas

A *Preprocesamiento*

GCC Se obtiene el codigo expandido después de la remoción de comentarios, agregado de código de los *headers* y la expansión de macros. El resultado es un archivo de extensión `.i`. La salida se muestra en la Fig. 4.4.

CLANG/LLVM El código extendido generado como salida del preprocesador en CLANG también es un archivo de extensión `.i` y se presenta en 4.5.

B *Analizador léxico y sintáctico*

GCC GCC no permite obtener los tokens válidos en forma de texto. Es un archivo interno.

CLANG/LLVM CLANG permite obtener los *tokens* válidos resultantes de la acción de par *lexer-parser*.

```
int 'int'  [StartOfLine]  Loc=<ejemplo.c:1:1>
```

Listing 4.6: CLANG/LLVM `tokens.c`. [8]

Se explica con detalle la primera línea (primer *token*) del código.

- **Int** define el tipo de *token* que define la palabra. **Int** es el nombre de la palabra en sí.
- ***StartOfLine*** es el *flag* de token que indica que el elemento está al inicio de cada línea de código o después de espacios en blanco al inicio de cada línea.
- **Loc** indica la ubicación en archivo de código fuente, línea de código dentro del archivo y número de caracter en la línea de código dada. En este caso, "int" está en el archivo `ejemplo.c`, en la línea número 1 e inicia en el caracter 1 de la línea 1.

El otro *flag* mostrado aquí es `LeadingSpace`, que indica que existe un espacio en blanco antes de dicho *token*.

C *Analizador semántico - fin del* frontend

GCC Se observa el árbol tipo AST. Se presenta el archivo formato `.ORIGINAL` entregado por GCC al activar la bandera *dump*. La letra "t" al lado del número de *dump* indica que pertenece al proceso de generación de GIMPLE (*Tree SSA*).

Se examina la línea *16* (`num_a`) del código `ejemplo.c.004t.original` [14]. La variable `num_a` está identificada con `@9` (ver línea 9 en código referenciado [14]). La variable apuntada por `@9`, es de tipo `@10` (*integer*) (líneas 17 a 19 en código [14]) y tiene valor inicial `@12` (3) (línea 22 en código [14]).

Es posible armar así el árbol hacia arriba y hacia abajo con mayor detalle de la función principal *main* así como más información sobre el tipo de variable *integer*.

CLANG/LLVM También CLANG provee un árbol de tipo AST. CLANG permite obtener la organización del árbol. Se observa que el formato de almacenamiento de información es distinto.

Se sigue el análisis del *token* (`num_a`). Referenciar para ello el archivo `ejemplo.ast` [15], líneas 16 a 29.

Se observa que el *token* está dentro de la función *main* de tipo *int* (*FunctionDecl*). El nodo *DeclStmt* y *VarDecl* define la existencia de declaración de variable llamada `num_a` de tipo `int`. *IntegerLiteral* expresa que el valor de la variable es un entero con valor "3".

D *Generador de código intermedio*

```
1  main ()
2  {
3    int D.1915;
4
5    {
6      int num_a;
7      int num_b;
8
9      num_a = 3;
10     num_b = 4;
11     num_b = num_a;
12     D.1915 = 0;
13     return D.1915;
14   }
15   D.1915 = 0;
16   return D.1915;
17 }
```

Listing 4.7: Código intermedio obtenido en GCC - `ejemplo.c.005t.gimple`. [9]

```
1   %1 = alloca i32, align 4
2   %2 = alloca i32, align 4
3   %3 = alloca i32, align 4
```

Listing 4.8: Asignación de variables en LLVM IR de CLANG/LLVM - `ejemplo.ll`. [10]

```
1   %2 = num_a = 3
2   %3 = num_b = 4
3   %4 = %2 = 3
4   %3 = num_b = %4
```

Listing 4.9: Pseudocódigo de la asignación de variables en LLVM IR - `ejemplo.ll`. [10]

GCC El código intermedio usado en GCC es GIMPLE. La letra "t" al lado del número de *dump* indica que pertenece al proceso de generación de GIMPLE (*Tree SSA*)

Se observa que el código generado en Listing 4.7 ya no sigue con la gramática dada por el lenguaje C. La función *main* define una variable entera de nombre `D.1915`, que es la variable que se va a retornar al fin de la función. En este caso, se devuelve 0 (dado por `return(0)`)

CLANG/LLVM El código intermedio de CLANG/LLVM es LLVM IR. [10] Otra vez, el cambio de formato es evidente. Ver Listing 4.8 y Listing 4.9.

Este compilador almacena información de la arquitectura a la que va orientada en el código intermedio (*target triple*) así como el *layout* de almacenamiento de datos en memoria dependiendo de la arquitectura que se debe seguir

(*data layout*). Luego, se define la función *main*, de tipo *integer* de 32 bits. Dentro de ella, se pasa a una declaración típica de tipo tres direcciones con variables según índice. Se crean y reservan memoria para tres variables de 32 bits.

Luego, se asigna el valor "3" y "4" a lo que sería `num_a` y `num_b` respectivamente. No se reusa la variable `num_b` para la copia de información sino que se crea una nueva (recordar que en SSA toda variable puede ser asignada una sola vez). Finalmente, se carga el valor de %2 (`num_a`) en una nueva variable %4 y se la guarda al ya existente %3 (`num_b`).

E Optimizador

GCC Se obtiene el código optimizado de formato RTL denominado `c.final` en GCC. [16] La letra `r` al lado del número de *dump* indica que pertenece al proceso de generación de RTL.

En la líneas 37 a 44 del código optimizado (`ejemplo.c.317r.final`) [16], se observa la definición de las variables `num_a` y `num_b` en un lenguaje parecido a *assembler*. Se divisa el uso de índices y modo de direccionamiento de bajo nivel. Luego, en las líneas 45 a 52 del código [16], se carga el valor *3* a `num_a` y *4* a `num_b`. Se guarda el valor de `num_a` en *stack* (posición: -8) y se hace una copia del valor *3* a la posición dada por `num_a`. Finalmente, se carga el valor de `num_a` y luego se hace un *set* (asignación) hacia `num_b`.

CLANG/LLVM NOTA: Se recuerda que la etapa de optimización forma parte del middle-end de LLVM. Por motivos de organización, se presenta en esta subsección.

Se expone el final del proceso de optimización en el *backend* de LLVM. Las instrucciones son muy parecidas a código *assembler*, usando ciertas instrucciones propias del *set* de ASM. Se pone foco en las líneas 1426 a 1450 del código optimizado [17]. Se asignan los valores de `num_a` y `num_b` usando la instrucción *MOV32mi* referenciando las posiciones de memoria con el par `RBP` (*extended base pointer*) y los índices (`RBP-8` para `num_a`, `RBP-4` para `num_b`). Se carga en `EAX` el valor de `num_a` (`RBP-8`) y luego, también se carga `EAX` en `num_b` (`RPB-4`).

III Código máquina

GCC y CLANG/LLVM Se observa que el código máquina obtenido por GCC (`ejemplo.s`, [11]) y por CLANG/LLVM (`ejemplo.s`, [18]) es idéntico en ambos casos. Se presenta el código ensamblador obtenido para GCC en el Listing 4.10.

```
1  main:
2  .LFB0:
3    .cfi_startproc
4    endbr64
5    pushq %rbp
6    .cfi_def_cfa_offset 16
7    .cfi_offset 6, -16
8    movq  %rsp, %rbp
9    .cfi_def_cfa_register 6
10   movl  $3, -8(%rbp)
11   movl  $4, -4(%rbp)
12   movl  -8(%rbp), %eax
13   movl  %eax, -4(%rbp)
14   movl  $0, %eax
15   popq  %rbp
16   .cfi_def_cfa 7, 8
17   ret
```

Listing 4.10: Código ensamblador generado por GCC - `ejemplo.s`. [11]

Capítulo 5

Código ejecutable

I Formatos de ejecutables en Unix

Los formatos de código ejecutable utilizados en Unix se presentan en la Tabla 5.1, así como sus principales características.

II Formato ELF

Por diseño, el formato ELF es **flexible**, **extensible** y **multiplataforma**. Compatible con diferentes *endianness* y tamaños de direcciones, por lo que no excluye ninguna unidad central de procesamiento (CPU) o arquitectura de conjunto de instrucciones en particular. Esto ha permitido que sea adoptado por muchos sistemas operativos diferentes en muchas plataformas de *hardware* diferentes. ELF ha sido elegido como **formato estándar** para archivos de tipo binario en sistemas Unix y casi-Unix en procesadores x86. No tiene extensión definida. [19]

A *Información básica de un archivo ELF*

Al usar el comando *file*, se puede ver el formato de cada archivo.

```
$ file simpleEjGCC
simpleEjGCC: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked,
    ↪ interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=
    ↪ ad6012cd2941aebbac623fe05b3501922a1ef126, for GNU/Linux 3.2.0, not stripped
```

Listing 5.1: Salida del comando *file* al ejecutable ELF `simpleEjGCC`.

Usando el comando `objdump -f`, se imprimen las cabeceras del archivo ELF.

```
    objdump -f simpleEjGCC

    simpleEjGCC:     file format elf64-x86-64
    architecture: i386:x86-64, flags 0x00000150:
    HAS_SYMS, DYNAMIC, D_PAGED
    start address 0x0000000000001040
```

Listing 5.2: Comando `dump` sobre el archivo ELF `simpleEjGCC`.

Tabla 5.1: Principales formatos de códigos ejecutables en Unix.

OUT	COFF	ELF
Assembler Output Format (OUT) es el formato más antiguo y clásico. Usa un *header* compacto y corto. Contiene tres segmentos: ▪ *.text* ▪ *.data* ▪ *.bss* con una tabla de simbolos y una tabla de *strings*	El formato Common Object File Format (COFF) implica un objeto de formato SVR3. El *header* está formado por una tabla de sección. Entonces, se puede tener más secciones aparte de *.text*, *.data* y *.bss*.	Executable and Linking Format (ELF) es un formato sucesor a COFF. Tiene múltiples secciones y puede tener valores de 32 o 64 bits. Por su flexibilidad y portabilidad es el formato estándar en Unix. Más detalle se presenta en la sección II.

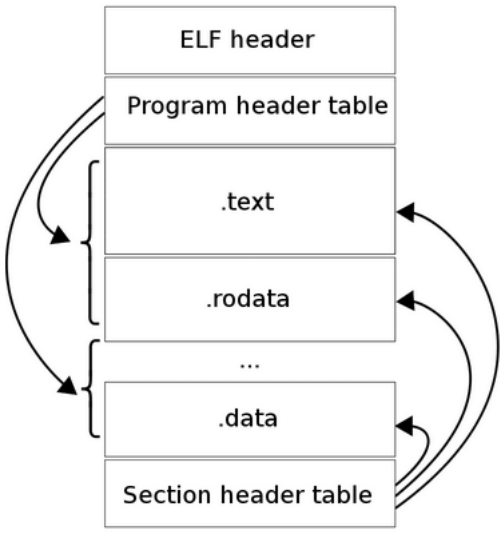


Figura 5.1: Estructura de un archivo ELF

B Estructura de un archivo ELF

1) Header *ELF*

Un *header* (cabecera, en español) tiene un largo de 32 *bytes*. Describe el archivo ELF en general. Tiene punteros a cada una de las secciones individuales que conforman el archivo. [20]

El *header* indica si es de 32 o 64 bits, *little* o *big endian*, muestra la versión ELF, el sistema operativo para el cual fue compilado y el *instruction set* del CPU.

Se lee el *header* del ELF con el comando `readelf` (presentación legible para humanos del *header* ELF). El Listing 5.3 muestra algunas líneas seleccionadas de la cabecera del ejecutable.

```
$ readelf -h simpleEjGCC
ELF Header:
Magic:  7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
Class:                             ELF64
OS/ABI:                            UNIX - System V
Entry point address:               0x1040
Start of program headers:          64 (bytes into file)
Start of section headers:          14608 (bytes into file)
```

```
Size of this header:               64 (bytes)
Size of program headers:           56 (bytes)
Number of program headers:         13
Size of section headers:           64 (bytes)
Number of section headers:         29
Section header string table index: 28
```

Listing 5.3: Lectura de la cabecera ELF usando el comando `readelf` sobre el archivo `simpleEjGCC`.

2) Diferencia entre secciones y segmentos

Existen dos vistas sobre un ELF. Uno usado para el *linker* y otro usado para la ejecución del programa. Las *secciones* esperan ser *linkeadas* en un ejecutable. Una o más secciones son mapeadas a *segmentos* dentro del ejecutable.

3) Segmentos

- Pueden contener 0 o más secciones.
- Contienen los datos que necesita el sistema operativo en forma dinámica para ejecutar el archivo.
- Indica al sistema operativo el lugar en memoria virtual en donde debe cargarse los datos que forman el segmento.
- Explicita los permisos de lectura, escritura y ejecución que tiene cada segmento.

4) Secciones

- Contienen datos estáticos útiles en el linkeo.
- Indica al *linker* si los datos de la sección son:
  - *Raw* data a cargar en memoria (Ej: secciones *.data* o *.text*).
  - *Metadata* con información sobre otras secciones, serán usadas por el *linker* y desaparecen en tiempo de ejecución (Ej: *.symtab, srttab*, etc).
- El *linker* coloca secciones dentro de un determinado segmento. Una o más secciones son mapeadas a segmentos formas de organizar el binario en áreas lógicas para comunicar información entre el compilador y el *linker*.

```
Section to Segment mapping:
Segment Sections...
    00
    01     .interp
    02     .interp .note.gnu.property .note.gnu.build-id .note.ABI-tag .gnu.hash .dynsym .
    ↪ dynstr .gnu.version .gnu.version_r .rela.dyn
    03     .init .plt .plt.got .text .fini
    04     .rodata .eh_frame_hdr .eh_frame
    05     .init_array .fini_array .dynamic .got .data .bss
```

Listing 5.4: Algunas líneas del mapeo entre secciones y segmentos del archivo ELF `simpleEjGCC`.

5) Tabla de headers *de programa*

- Contiene los segmentos del archivo.
- Indica al sistema operativo como crear la imagen del proceso, es decir, la ejecución del programa.

Los archivos usados para ejecutar el programa deben tener una tabla de *header* de programa. Archivos *reubicables* no necesitan de uno.

Variables reubicables Archivos que tienen espacio reservado (rellenos con espacios en blanco) y que son llenados una vez que la dirección de los mismos sea conocida.

```
$ readelf -l simpleEjGCC
Program Headers:
Type    Offset  VirtAddr PhysAddr FileSiz MemSiz   Flags  Align
PHDR    0x00040 0x00040  0x00040  0x002d8 0x002d8  R      0x8
INTERP  0x00318 0x00318  0x00318  0x0001c 0x0001c  R      0x1
            [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD    0x00000 0x00000  0x00000  0x005c8 0x005c8  R      0x1000
LOAD    0x01000 0x01000  0x01000  0x001d5 0x001d5  R E    0x1000
LOAD    0x02000 0x02000  0x02000  0x00130 0x00130  R      0x1000
```

Listing 5.5: Algunos *headers* de segmentos del archivo ELF `simpleEjGCC`.

6) Tabla de headers *de sección*

- Contiene información de las distintas secciones del ELF.
- Cada sección tiene una entrada en la tabla.

Entrada Muestra la cantidad de *section headers*, el tamaño de cada sección, el tipo de sección y la dirección y el *offset* en memoria.

```
$ readelf -S simpleEjGCC
There are 29 section headers, starting at offset 0x3910:

Section Headers:
[Nr] Name      Type      Address Offset Size   EntSize Flags Link  Info  Align
[14] .text     PROGBITS  001040  01040  00185  00000   AX      0     0     16
[16] .rodata   PROGBITS  002000  02000  00004  00004   AM      0     0     4
[23] .data     PROGBITS  004000  03000  00010  00000   WA      0     0     8
[24] .bss      NOBITS    004010  03010  00008  00000   WA      0     0     1
```

Listing 5.6: Algunos *headers* de sección del archivo ELF `simpleEjGCC`.

Capítulo 6

Inserción de información en archivo ELF

I Secciones ELF en donde colocar payload

Después de examinar las distintas secciones y realizar varias pruebas, se presentan tres posibles lugares en donde colocar la información planteada.

- Sección `.comment`.
  - Al inicio, contiene información sobre formato de compilación de GCC.
- Sección *Read only data* `.rodata`.
  - Sección ya creada por el compilador. Guarda los *strings* y las constantes definidas en el código del programa.
- Crear una nueva sección.

A *Sección `.comment`*

La sección `.comment` forma parte de las secciones predefinidas en un archivo ELF. Al terminar el proceso de compilación y enlace de objetos, GCC coloca información de compilación en esta sección. Por lo tanto, no es vacía.

Lectura en formato *string* de la sección `.comment`.

```
    $ readelf -p .comment simpleEjGCC
String dump of section '.comment':
  [     0]  GCC: (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0
```

Listing 6.1: Sección de comentario dentro del ELF `simpleEjGCC`.

1) Inline assembly

Se coloca una cadena de caracteres en la sección `.comment` usando código ensamblador (*inline assembly*) dentro del código fuente del programa C. [21]

```
int main() {
  int num_a = 3;
  int num_b = 4;

  num_b = num_a;

```

```
  __asm__(".section .comment\n\t"
        ".string \"Hello World con inline asm\"\n\t"
        ".section .text");

  return 0;
}
```

Listing 6.2: Lenguaje ensamblador para insertar datos en sección `.comment` en `simpleEjInline`.

El resultado en la modificacion de la sección `.comment` muestra la información de compilación anteriormente agregada por GCC y la información agregada en esta demostración.

```
    $ readelf -p .comment simpleEjInlineGCC

String dump of section '.comment':
  [     0]  GCC: (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0
  [    2a]  Hello World con inline asm
```

Listing 6.3: Lectura de la sección `.comment` del archivo `simpleEjInlineGCC`.

2) Objcopy

Empleo de la herramienta *Objcopy* para agregar datos dentro de una sección del ejecutable ELF. Para ello, es necesario tener un ELF ya existente.

```
readelf -p .comment simpleEjGCC

String dump of section '.comment':
  [     0]  GCC: (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0

objcopy --update-section .comment=comentario.txt simpleEjGCC simpleComentGCC
readelf -p .comment simpleComentGCC

String dump of section '.comment':
  [     0]  Este es un comentario
```

Listing 6.4: Lectura de la sección `.comment` del archivo `simpleEjInlineGCC` después de realizar copia de la información.

Aclaraciones de este método:

- Se realiza post-compilación.
- Se crea un nuevo ELF con la sección actualizada. Se toma el ELF original, se copian los datos, se adjunta la nueva información a la original y ésta es colocada en un nuevo ELF.

3) Atributos de C

La palabra reservada `__attribute __` permite activar atributos especiales de variables o estructuras, funciones y tipos. [22]

La función atributo *`section`* permite insertar código en las distintas secciones de la imagen del archivo.

Agrego comentario a la sección `.comment` con `atributte`

```
    int main() {

        int num_a = 3;
```

```
        int num_b = 4;

        num_b = num_a;

        return 0;
    }

    const char str[] __attribute__((section(".comment#"))) = "Hello ELF";
```

Listing 6.5: Inserción de payload en sección `.comment` usando `atributte`.

Se utiliza el símbolo # al lado del nombre de la sección para evitar *warning* de *assembler*. [23]

```
    $readelf -p .comment simpleEjGCC_ATR

String dump of section '.comment':
  [     0]  GCC: (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0
  [    30]  Hello ELF
  [    3a]  GCC: (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0
```

Listing 6.6: Sección `.comment` usando `attribute` para insertar payload.

Aclaraciones de éste método:

- ERROR: Al emplear este método, no se borra línea del compilador ya existente en la sección. Se inserta una nueva y vuelve a repetir la línea de compilador.

B *Sección* Read only data

En formatos ELF, las constantes de *string* definidas en el código se almacenan en la sección `.rodata` del ELF. [24]

```
#include <stdio.h>

int main() {
  const char *str = "Comentario dentro de read-only section";
  printf("Esto es un printf");
  return 0;
}
```

Listing 6.7: Inserción de payload en sección `.rodata` - `cod_rodata.c`.

Se puede ver que en la seccion `.rodata` se encuentran ambos *strings*.

```
$ readelf -p .rodata ejStr_rodataGCC

String dump of section '.rodata':
  [     8]  Comentario dentro de read-only section
  [    2f]  Esto es un printf
```

Listing 6.8: Sección `.rodata` con payload agregado.

Aclaraciones de éste método :

- Si declaramos otras constantes de *string* distintas, van a estar también presentes en la sección, entonces, habría que separarlas luego.

C *Nueva sección creada*

Ésta solución es la que más ventajas trae a la hora de customización. Implica crear una nueva sección llamada `.metadata` con la declaración `attribute`. La elección del nombre de la nueva sección para este trabajo, responde a la naturaleza de los datos a agregar: metadatos. Al ser una nueva sección, no contiene datos al momento de insertar la información requerida. Por ello, se evita mezclar y manipular datos útiles en el programa y reservar sólo la sección al *payload* de interés. Además, el contenido de esa sección puede ser pasado a través de la terminal al compilador. [25]

```
#include <stdio.h>

int main(int argc, char *argv[]) {

  int num_a = 3;
  int num_b = 4;

  num_b = num_a;
  printf("Num a: %d - Num b: %d\n", num_a, num_b);

  return 0;
}

  #ifdef COMENTARIO
    const char str[] __attribute__((section(".metadata#"))) = COMENTARIO;
  #else
  #endif
```

Listing 6.9: Inserción de payload en sección `.metadata` en ejecutable `macrosCommentGCC`.

El programa es compilado utilizando un *Makefile*. Ver 6.10.

```
  $ make MENSAJE="Comentario desde la terminal"

```

Listing 6.10: Comando de compilación para pasar el *string* (*payload*) como variable de entorno al compilador.

Finalmente, una vez obtenido el archivo ejecutable de tipo ELF, se puede observar los datos agregados y su salida en la sección creada tal como lo se indica en 6.11.

```
  $  readelf -p .metadata macrosCommentGCC

  String dump of section '.metadata':
    [     0]  Comentario desde la terminal
```

Listing 6.11: Contenido de la sección `.metadata`.

Todos los códigos presentados pueden ser accedidos desde el repositorio de trabajo. [26]

Capítulo 7

Lectura de metadata insertada en archivo ELF

Realizada la inserción de la cadena de caracteres (*string*) en el ejecutable ELF, se procede a la lectura de la misma empleando un programa propio. Este programa ha sido creado para poder extraer0 y guardar el *string* para futuras funcionalidades.

I Lectura ELF

Este programa de espacio de usuario toma como entrada el archivo ELF obtenido anteriormente.

```
1     FILE* elf_fp;
2
3     elf_fp = fopen(ELF_PATH, "r");
4     if(elf_fp == NULL){
5         perror("Error al leer ELF - ");
6         printf("Error %d: %s\n", errno, strerror(errno));
7         exit(EXIT_FAILURE);
8     }
```

Listing 7.1: Carga del archivo ELF en código `lecturaELF.c`.

Se extrae la cabecer del archivo ELF (*ELF header*). Luego, almacena la tabla de símbolos, o también llamada tabla de cabecera de secciones (*section headers table*)

```
1     symbol_table = malloc(elf_header.e_shentsize*elf_header.e_shnum);   // Reserva memoria para
      ↪  alojar tabla de simbolos ELF
2
3     fseek(elf_fp, elf_header.e_shoff, SEEK_SET);    // Indice posicion apunta al inicio de
      ↪ header ELF
4
5     fread(symbol_table, 1, elf_header.e_shentsize*elf_header.e_shnum, elf_fp);  // Cargo todas
      ↪ las entradas de tabla de simbolos
```

Listing 7.2: Lectura y almacenamiento de la tabla de símbolos en `lecturaELF.c`.

Se copia la tabla de strings, que contiene los nombres de las secciones. Itero sobre cada entrada, comparando el nombre de la sección buscada.

```
1     buff = malloc(sh_table[eh.e_shstrndx].sh_size); // Reservo memoria para alojar la entrada a
      ↪  seccion que busco
2
3     if(buff != NULL)
4     {
```

```
5         fseek(fd, sh_table[eh.e_shstrndx].sh_offset, SEEK_SET); // Indice apunta a la tabla de
  ↪ strings
6         fread(buff, 1, sh_table[eh.e_shstrndx].sh_size, fd);    // Copio la tabla de strings
7     }
8
9     sh_str = buff;  //
10
11    for(i=0; i<eh.e_shnum; i++) // Recorro todas las entradas del header de la tabla
12    {
13        if(strcmp(nombreSeccion, (sh_str + sh_table[i].sh_name)) == 0)  // Comparo si es la
  ↪ seccion que yo quiero
14        {
15            printf("-- Seccion %s encontrada en offset 0x%08x y de tamanio %ld --\n",
  ↪ nombreSeccion, (unsigned int)sh_table[i].sh_offset, sh_table[i].sh_size);
16            break;
17        }
18    }
```

Listing 7.3: Lectura y almacenamiento de la tabla de *strings* en código `lecturaELF.c`.

Una vez que la comparación del nombre de sección es exitosa, extraigo el número de índice de la sección dentro del archivo ELF. El índice se utiliza para guardar ubicar y guardar la sección. Se copian los datos de la sección indicada por el índice. Finalmente, se imprime por pantalla la cadena de caracteres y se verifica que es la misma que ha sido insertada previamente.

```
1         char *datos_seccion = malloc(sh_table[i].sh_size); // Reservo memoria para almacenar
  ↪ string buscado
2         fseek(fd, sh_table[i].sh_offset, SEEK_SET); // Ubico indice al inicio de la seccion
  ↪ encontrada
3         fread(datos_seccion, 1, sh_table[i].sh_size, fd);  // Copio todos los datos de la
  ↪ seccion
4         printf("//// Contenido de seccion: ////\n%s\n", datos_seccion);
```

Listing 7.4: Almacenamiento e impresión de los datos que contiene la sección buscada. Código del programa en `lecturaELF.c`.

Al ejecutar el programa, la impresión de los datos de la sección `.metadata` se realiza por pantalla, tal como se muestra en

```
1     $  ./lecturaELF
2
3     -- Seccion .metadata encontrada en offset 0x00003040 y de tamanio 30 --
4     //// Contenido de seccion: ////
5     Nueva metadata desde terminal
6
```

Listing 7.5: Ejecución del programa `lecturaELF`.

Capítulo 8

Conclusión

Es posible observar que la compilación empleando GCC como CLANG/LLVM respeta las fases estándar de todo compilador moderno. Las diferencias que se encuentran residen en los formatos de código interno que utilizan, en especial, código intermedio de *middle-end*: GIMPLE, para GCC, y LLVM IR, para CLANG/LLVM. A su vez, podemos observar que el código en *assembler* final es el mismo en esencia, independiente del compilador usado. Dichas diferencias residen en el nombre de variables y otros *tags* así como las direcciones de cada elemento (distintas por obvias razones). CLANG/LLVM permite una mayor flexibilidad a la hora de definir optimizaciones y formatos de códigos internos a emitir. Sin embargo, para este cometido, no se ha desplegado el máximo provecho de dichas optimizaciones.

Después de haber investigado y entendido aspectos del formato de archivo ejecutable ELF, se han manejado distintos escenarios en los que cumple la función de insertar información útil dentro del ejecutable. El método preferido ha sido el planteado en la sección C. La creación de una sección nueva implica obtener una sección sin información previa. De tal manera, se evita recorrer la sección para encontrar una línea de *payload* (y con ello tener que especificar un formato detectable en la búsqueda de la línea). Además, no se mezcla el *payload* agregado con variables del programa (como sucedía en las otras opciones). El pasaje de datos mediante terminal utilizando el flag `-D` permite el ingreso del *payload* en el momento de escribir el comando de compilación por parte del usuario, y así, evitar tener que modificar el código fuente. Finalmente, el desarrollo del programa `lecturaELF` (presentado en la sección I), empleado para extraer e imprimir los datos ingresados, ha permitido verificar la correcta inserción.

El mecanismo inserción-extración servirá como punto inicial para las futuras implementaciones a plantear para el desarrollo del Trabajo Integrador de la carrera.

Bibliografía

[1] A. V. Aho and A. V. Aho, Eds., *Compilers: principles, techniques, & tools*, 2nd ed. Boston: Pearson/Addison Wesley, 2007.

[2] "Three address code in Compiler," May 2018. [Online]. Available: https://www.geeksforgeeks.org/three-address-code-compiler/ (Last accessed on 2021-11-17).

[3] C. Hock-Chuan, "GCC and Make - A Tutorial on how to compile, link and build C/C++ applications." [Online]. Available: https://www3.ntu.edu.sg/home/ehchua/programming/cpp/gcc_make.html (Last accessed on 2021-11-17).

[4] P. H. Dave and H. B. Dave, *Compilers: Principles and Practice*. Pearson India, 2012.

[5] J. Cancinos and J. Gonzalez, "Codigo fuente - ejemplo.c." [Online]. Available: https://github.com/josemacan/PPSCompiladoresUNC/blob/master/PPS/GCCvsCLANGLLVM/GCC/ejemplo.c (Last accessed on 2021-11-17).

[6] J. Cancinos and J. Gonzalez, "Código resultante del preprocesador en gnu gcc - ejemplo.i." [Online]. Available: https://github.com/josemacan/PPSCompiladoresUNC/blob/master/PPS/GCCvsCLANGLLVM/GCC/ejemplo.i (Last accessed on 2021-11-17).

[7] J. Cancinos and J. Gonzalez, "Código resultante del preprocesador en clang/llvm - ejemplo.i." [Online]. Available: https://github.com/josemacan/PPSCompiladoresUNC/blob/master/PPS/GCCvsCLANGLLVM/CLANGLLVM/ejemplo.i (Last accessed on 2021-11-17).

[8] J. Cancinos and J. Gonzalez, "Clang/llvm - tokens.txt." [Online]. Available: https://github.com/josemacan/PPSCompiladoresUNC/blob/master/PPS/GCCvsCLANGLLVM/CLANGLLVM/tokens.txt (Last accessed on 2021-11-17).

[9] J. Cancinos and J. Gonzalez, "Gcc - ejemplo.c.005t.gimple." [Online]. Available: https://github.com/josemacan/PPSCompiladoresUNC/blob/master/PPS/GCCvsCLANGLLVM/GCC/ejemplo.c.005t.gimple (Last accessed on 2021-11-17).

[10] J. Cancinos and J. Gonzalez, "Clang/llvm - readable llvm ir - ejemplo.ll." [Online]. Available: https://github.com/josemacan/PPSCompiladoresUNC/blob/master/PPS/GCCvsCLANGLLVM/CLANGLLVM/ejemplo.ll (Last accessed on 2021-11-17).

[11] J. Cancinos and J. Gonzalez, "Gcc- ejemplo.s." [Online]. Available: https://github.com/josemacan/PPSCompiladoresUNC/blob/master/PPS/GCCvsCLANGLLVM/GCC/ejemplo.s (Last accessed on 2021-11-17).

[12] W. Von Hagen, *The definitive guide to GCC*, 2nd ed. Berkeley, CA: Apress, 2006.

[13] J. Cancinos and J. Gonzalez, "Repositorio de trabajo." [Online]. Available: https://github.com/josemacan/PPSCompiladoresUNC/tree/master/PPS/GCCvsCLANGLLVM (Last accessed on 2021-11-17).

[14] J. Cancinos and J. Gonzalez, “Gcc - ejemplo.c.004t.original.” [Online]. Available: https://github.com/josemacan/PPSCompiladoresUNC/blob/master/PPS/GCCvsCLANGLLVM/GCC/ejemplo.c.004t.original (Last accessed on 2021-11-17).

[15] J. Cancinos and J. Gonzalez, “Clang/llvm - ejemplo.ast.” [Online]. Available: https://github.com/josemacan/PPSCompiladoresUNC/blob/master/PPS/GCCvsCLANGLLVM/CLANGLLVM/ejemplo.ast (Last accessed on 2021-11-17).

[16] J. Cancinos and J. Gonzalez, “Gcc - ejemplo.c.317r.final.” [Online]. Available: https://github.com/josemacan/PPSCompiladoresUNC/blob/master/PPS/GCCvsCLANGLLVM/GCC/ejemplo.c.317r.final (Last accessed on 2021-11-17).

[17] J. Cancinos and J. Gonzalez, “Clang/llvm - machineinstruct.txt.” [Online]. Available: https://github.com/josemacan/PPSCompiladoresUNC/blob/master/PPS/GCCvsCLANGLLVM/CLANGLLVM/machineinstruct.txt (Last accessed on 2021-11-17).

[18] J. Cancinos and J. Gonzalez, “Clang/llvm - ejemplo.s.” [Online]. Available: https://github.com/josemacan/PPSCompiladoresUNC/blob/master/PPS/GCCvsCLANGLLVM/CLANGLLVM/ejemplo.s (Last accessed on 2021-11-17).

[19] “Drysdale, david. “how programs get run: Elf binaries.” 2015.” [Online]. Available: https://lwn.net/Articles/631631/ (Last accessed on 2021-11-17).

[20] “Understanding the elf file format, linuxhint.” [Online]. Available: https://linuxhint.com/understanding_elf_file_format/ (Last accessed on 2021-11-17).

[21] “What is the significance of ".comment"section in elf?, quora.” [Online]. Available: https://www.quora.com/What-is-the-significance-of-comment-section-in-ELF (Last accessed on 2021-11-17).

[22] “Declaring attributes of functions, gnu.” [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc-3.2/gcc/Function-Attributes.html (Last accessed on 2021-11-17).

[23] “Assembler warning with gcc warning when placing data in .text, stackoverflow.” [Online]. Available: https://stackoverflow.com/questions/58455300/assembler-warning-with-gcc-warning-when-placing-data-in-text (Last accessed on 2021-11-17).

[24] “Add ascii comments to a c binary?, stackoverflow.” [Online]. Available: https://stackoverflow.com/questions/32740668/add-ascii-comments-to-a-c-binary (Last accessed on 2021-11-17).

[25] “How to define a string literal in gcc command line?, stackoverflow.” [Online]. Available: https://stackoverflow.com/questions/2410976/how-to-define-a-string-literal-in-gcc-command-line (Last accessed on 2021-11-17).

[26] Cancinos, Jose and Gonzalez, Julian, “Repositorio de trabajo.” [Online]. Available: https://github.com/josemacan/PPSCompiladoresUNC/tree/master/PPS/ELF/GCC (Last accessed on 2021-11-17).